

Documento de Levantamento de Requisitos Sistema CRUD Web com *** | FastAPI

1. Visão Geral do Projeto

Desenvolver uma aplicação web CRUD utilizando Flask | FastAP,segundo padrões modernos do mercado de trabalho

2. Objetivos Técnicos

Camada	Tecnologia
Backend Web	***
API	FastAPI
Banco de Dados	PostGreSQL
ORM	SQLAlchemy
Autenticação	JWT
DEPLOY	AWS

Aplicar arquitetura moderna de backend, APIs Restful, autenticação JWT, validações , escalabilidade e deploy em nuvem.

3. Tecnologias Envolvidas

Python 3.14

FastAPI
PostgreSQL
API Whatsapp

4. Requisitos Funcionais

RF01 – Requisição dos Dados do Cliente
RF02 – Requisição de Data e Hora
RF03 – Cadastro, Consulta, Alteração e Exclusão de Agendamentos
RF04 – Validação dos Dados Informados
RF05 – Consulta de Agendamentos
RF06 – Confirmação de Cancelamento de Agendamento

5. Requisitos não Funcionais

RNF01 – Desempenho
RNF02 – Disponibilidade
RNF03 – Segurança
RNF04 – Integridade dos Dados
RNF05 – Usabilidade
RNF06 – Confiabilidade
RNF07 – Backup e Recuperação
RNF08 – Compatibilidade
RNF09 – Escalabilidade
RNF10 – Manutenibilidade
RNF11 – Validação de Dados
RNF12 – Auditoria

6. Arquitetura Esperada

Frontend -> Flask -> FastAPI -> Services -> Repositories -> PostgreSQL

7. Estrutura Recomendada

project/
app/
tests/
migrations/
docker/
[main.py](#)

8. Regras de Negócio

RN01 – Identificação do Cliente
RN02 – Criação de Agendamento
RN03 – Validação de Data e Horário
RN04 – Alteração de Agendamento
RN05 – Cancelamento de Agendamento
RN06 – Consulta de Agendamentos
RN07 – Status do Agendamento
RN08 – Conflito de Agendamento
RN09 – Integridade dos Dados
RN10 – Confirmação via WhatsApp
RN11 – Auditoria
RN12 – Disponibilidade de Horários

9. Critérios de Aceitação

- Cadastro de Cliente Funcional
- CRUD operacional
- Validação de Dados
- Confirmação Via Whatsapp
- API documentada

10. Boas Práticas Utilizadas no Mercado

- Clean Architecture
- API RESTful
- Docker e CI/CD
- Testes Automatizados
- Logging e monitoramento

11. Requisitos de Infraestrutura

Banco: PostgreSQL
Deploy: AWS / Render / Railway
Proxy: Nginx
Versionamento: Git/Github

12. Conclusão

O documento estabelece os requisitos necessários para desenvolvimento de uma aplicação moderna, segura, escalável e alinhada ao mercado